

WhatsApp LangChain — Documentação Detalhada do Projeto

Referência arquivo por arquivo. Para visão geral rápida veja o [README.md](#); para arquitetura conceitual veja [docs/ARCHITECTURE.md](#). Este documento existe para explicar **o que cada arquivo faz e como eles se encaixam**.

Visão geral em uma frase

Um bot de WhatsApp orientado a agentes LangGraph, com **API** (FastAPI) que recebe webhooks do Twilio e enfileira mensagens no **PostgreSQL**, um **Worker** assíncrono que consome a fila, processa mídia, executa o agente e responde via Twilio, e um **Admin Panel** (Next.js) para inspecionar conversas e métricas. Redis cuida do rate limit distribuído.

```
WhatsApp/Twilio -> API (/webhook/twilio) -> PostgreSQL (message_queue)
                                     -> Worker -> LangGraph Agent
                                     -> PostgreSQL (response, conversation)
```

1. Raiz do repositório

`pyproject.toml`

Define o pacote Python `whatsapp-langchain` (build via `hatchling`, código em `src/whatsapp_langchain`). Lista as dependências de runtime: `langchain / langgraph` (agentes), `fastapi / uvicorn` (API), `psycpg` (Postgres async), `pydantic-settings` (config), `structlog` (logs), `twilio` (SDK), `redis`, `itsdangerous` (cookies assinados). Dependências de dev (`ruff`, `pyright`, `pytest`, `fakeredis`) ficam no extra `dev`; `locust` fica no extra `stress`. Também configura lint (`ruff`), type-checking (`pyright`) e pytest (`asyncio_mode = "auto"`, marker `docker_demo`).

`uv.lock`

Lockfile gerado pelo `uv` (gerenciador de pacotes Python) — trava versões exatas de todas as dependências transitivas para builds reproduzíveis.

langgraph.json

Manifesto usado pelo `langgraph dev` (LangGraph Studio/CLI). Registra o grafo `rhawk_assistant` apontando para `graph.py:graph` e carrega variáveis de `.env`. É o que permite abrir o Studio local para depurar o agente isoladamente, sem subir API/Worker.

Makefile

Ponto de entrada de todos os comandos operacionais do projeto (`make help` lista tudo). Grupos principais:

- **Setup/Dev:** `setup` (cria `.venv` com `uv`), `dev` (LangGraph Studio), `db` / `api` / `worker` / `frontend` (rodam cada serviço isoladamente fora do Docker).
- **Docker:** `up` / `down` / `reset` / `logs` (stack completa via `docker compose`).
- **Qualidade:** `lint`, `format`, `fix`, `typecheck`, `check`, `ci`.
- **Testes:** `test`, `test-x`, `test-v`, `test-live` (chamadas reais ao OpenRouter), `test-demo` (requer Docker), `test-flows`.
- **Stress:** `stress` / `stress-headless` (Locust contra a stack local).
- **Operação:** `backup` (dump do Postgres), `clean` (limpa `__pycache__`).

.env.example

Template de todas as variáveis de ambiente do projeto, com comentários explicando cada uma e a ordem recomendada de preenchimento (OpenRouter → Twilio → rate limit → admin auth → deploy). Copiado para `.env` (não versionado) via `make setup` / `cp .env.example .env`. Espelha 1:1 os campos de `shared/config.py`.

.env, .gitignore

`.env` é a configuração local real (não deve ser commitada — contém segredos). `.gitignore` garante isso e ignora artefatos de build (`.venv`, `__pycache__`, `.next`, etc).

README.md

Documento de entrada do projeto: o que é, fase atual (Fase 4 concluída), diagrama de arquitetura, quick start (`make setup` → `make up` → testar webhook), estrutura de pastas e roadmap por fases. Em português, com foco didático em "arquitetura de sistemas ao redor de um agente".

LICENSE

Licença "TOPHAWKS Community License" — uso restrito a membros da comunidade TOPHAWKS (não é uma licença open-source padrão).

`ffmpeg.exe`, `ffprobe.exe`, `railway.api.toml`, `railway.db.toml`, `railway.worker.toml`

Arquivos **não rastreados pelo git** (aparecem como `??` no `git status`) — não fazem parte do histórico do projeto. Os `.exe` parecem binários do FFmpeg deixados localmente (não usados por nenhum código-fonte lido); os `railway.*.toml` sugerem configuração de deploy na Railway (serviços `api/db/worker`) feita localmente, mas ainda não versionada.

2. Infraestrutura Docker

`docker-compose.yml`

Orquestra o stack de desenvolvimento: `db` (Postgres com pgvector, build local), `redis` (rate limit distribuído), `api` (FastAPI), `worker` (consumidor da fila) e `frontend` (Next.js). `api` e `worker` dependem de `db` / `redis` saudáveis (`healthcheck`) e recebem `DATABASE_URL` / `REDIS_URL` apontando para os nomes dos serviços internos (`db`, `redis`) em vez de `localhost`.

`docker-compose.prod.yml`

Override de produção (usado com `-f docker-compose.yml -f docker-compose.prod.yml`). Adiciona `restart: unless-stopped` e limites de CPU/memória a todos os serviços, e introduz o serviço `proxy` (Caddy) que expõe as portas 80/443 e emite TLS automático via Let's Encrypt usando a variável `DOMAIN`.

`Dockerfile.api`

Imagem da API: Python 3.12-slim, instala dependências com `uv pip install --system .`, copia `src/` e `db/` (migrações), roda como usuário não-root (`appuser`), expõe 8000 e define `HEALTHCHECK` batendo em `/health`. Comando final: `uvicorn whatsapp_langchain.server.main:app`.

`Dockerfile.worker`

Mesma base da API (Python 3.12-slim + `uv`), mas sem exposição de porta/healthcheck HTTP — roda `python -m whatsapp_langchain.worker.main`, o loop infinito de consumo da fila.

`Dockerfile.db`

Dockerfile do Postgres com a extensão `pgvector` pré-instalada (necessária para o `store` semântico de memória do LangGraph). Não pôde ser lido aqui por conter conteúdo binário/formatação não padrão para o Read tool, mas é referenciado por `docker-compose.yml` como build local do serviço `db`.

Caddyfile

Configuração do proxy reverso de produção. Roteia `/api/*`, `/webhook/*` e `/health` para o serviço `api:8000`; tudo o mais (o Admin Panel) vai para `frontend:3000`. O domínio vem de `{ $DOMAIN }` (variável de ambiente), e o Caddy cuida da emissão automática de certificado TLS.

frontend/Dockerfile

Build multi-stage do Next.js: `deps` (instala `node_modules`), `builder` (`npm run build`, gerando saída `standalone`), `runner` (imagem final mínima, usuário não-root `nextjs`, copia apenas `.next/standalone` + `.next/static` + `public`). Expõe 3000 com healthcheck em `/login`.

frontend/.dockerignore

Evita copiar `node_modules`, `.next` e afins para o contexto de build do Docker (usa cache de camadas do multi-stage acima).

3. Backend Python — `src/whatsapp_langchain/`

Pacote instalável (`pip install -e .`) com 4 subpacotes: `agents/`, `server/`, `worker/`, `shared/`.

`__init__.py`

Define `__version__ = "0.1.0"`, importado por `server/routes/health.py` para reportar a versão no `/health`.

`src/whatsapp_langchain.egg-info/`

Metadados gerados automaticamente pela instalação editável (`pip install -e .`) — `PKG-INFO`, `SOURCES.txt`, `requires.txt`, `top_level.txt`. Não é código-fonte, é artefato de build.

3.1 `shared/` — infraestrutura compartilhada entre API e Worker

`shared/config.py`

O coração da configuração: uma classe `Settings` (Pydantic `BaseSettings`) que lê `.env` /variáveis de ambiente e tipa **tudo** que o projeto precisa — URL do banco, porta, Twilio (inbound/outbound), rate limit HTTP e de LLM, Redis, autenticação do admin (usuário/senha/cookie), debounce, modelos OpenRouter (chat, mídia, embeddings), estratégia de contexto (`trim` / `summarize` / `none`) e memória semântica. Exporta um singleton `settings` importado em praticamente todos os outros módulos — garante que a configuração seja consistente em toda a aplicação, sem leituras dispersas de `os.getenv`.

shared/db.py

Gerencia o **pool de conexões PostgreSQL** (`psycopg_pool.AsyncConnectionPool` , singleton `get_pool()` / `close_pool()`) e o ciclo de vida do LangGraph:

- `run_migrations()` : aplica arquivos SQL de `db/migrations/` que ainda não constam na tabela de controle `_migrations` .
- `resolve_store_index_config()` : monta a config de embeddings (OpenRouter) usada pelo `AsyncPostgresStore` .
- `open_checkpointer()` / `open_store()` : abrem o checkpointer (`AsyncPostgresSaver`) e o store vetorial (`AsyncPostgresStore` , condicionado a `MEMORY_ENABLED`) com `AsyncExitStack` , para ciclo de vida explícito (abertos uma vez no boot da API/Worker, fechados no shutdown) — evita reabrir conexões a cada mensagem.
- `bootstrap_langgraph_schema()` : roda `.setup()` do checkpointer/store no startup, garantindo que as tabelas do LangGraph existam antes do primeiro request (sem criação lazy).
- `check_db_health()` : usado pelo endpoint `/health` .
- `_resolve_migrations_dir()` : resolve o caminho de `db/migrations` tanto em dev local (`src/...`) quanto em produção Docker (`/app/db/migrations`).

shared/models.py

Modelos **Pydantic** que atravessam as fronteiras entre módulos (API ↔ banco ↔ Worker):

- `MessageStatus` (enum `queued` / `processing` / `done` / `failed`).
- `MessageQueue` : espelha a tabela `message_queue` (telefone, agente, `thread_id` , mídia, status, tentativas, lease, resposta/erro).
- `Conversation` : espelha `conversations` (agregado por `phone_number` + `agent_id`).
- `TwilioWebhookPayload` : shape do form-data que o Twilio envia (`MessageSid` , `From` , `To` , `Body` , `NumMedia` , `MediaUrl0` , `MediaContentType0`).
- `EnqueueResult` : retorno de `enqueue_or_buffer` (id da mensagem + se foi bufferizada por debounce).

shared/queue.py

Toda a lógica de fila em PostgreSQL, usada pela API (produtor) e pelo Worker (consumidor):

- `enqueue_or_buffer()` : insere mensagem nova ou concatena com uma pendente do mesmo `phone+agent` (**debounce**). Mídia nunca faz debounce — ao chegar mídia, qualquer texto pendente é "flushado" (processado antes) para preservar a ordem. Usa `pg_advisory_xact_lock` (hash SHA-256 de `thread_id`) para serializar condições de corrida entre requisições concorrentes do mesmo usuário.
- `claim_next()` : consulta atômica com `FOR UPDATE SKIP LOCKED` que reserva a próxima mensagem elegível (`queued` com `process_after` vencido, ou `processing` com lease expirado) para um worker — permite múltiplos workers concorrentes sem duplicar processamento. Também recupera

mensagens presas (`processing` com lease vencido e sem tentativas restantes) marcando-as como `failed` .

- `mark_done()` / `mark_failed()` : finalizam o ciclo de vida da mensagem. `mark_failed` decide entre **retry com backoff** (`attempts * 5` segundos) ou falha definitiva, conforme `max_attempts` .
- `upsert_conversation()` : mantém a tabela `conversations` atualizada (usada pelo Admin Panel).

`shared/llm.py`

Factory central para criar instâncias de `ChatOpenAI` (apontando para o OpenRouter) com **rate limiting** via `InMemoryRateLimiter` do LangChain. Os limiters são cacheados por combinação de (`requests_per_second`, `max_bucket_size`) para que múltiplas chamadas ao mesmo tipo de modelo compartilhem o mesmo "balde" (throughput real do processo, não por chamada individual). Usada pelo agente principal e pelo middleware de summarize.

`shared/observability.py`

Configura `structlog` para todo o projeto: em dev, saída colorida (`ConsoleRenderer`); em produção (`LOG_JSON=true`), JSON estruturado. Reduz ruído de bibliotecas externas (`uvicorn.access` , `httpx` ficam em `WARNING`).

`shared/redis_client.py`

Client Redis assíncrono singleton (mesmo padrão de `shared/db.py` : `get_redis()` / `close_redis()` no lifespan). Usado exclusivamente pelo rate limit distribuído da API (`server/dependencies.py`), permitindo que múltiplas réplicas da API compartilhem o mesmo contador por telefone.

3.2 `agents/` — catálogo de agentes, middleware e tools

`agents/loader.py`

Carregador **dinâmico** de agentes por `agent_id` . `load_graph(agent_id, checkpoint, store)` importa `agents.catalog.{agent_id}.agent` via `importlib` e chama sua função `build_graph()` . `list_agents()` escaneia `agents/catalog/` por diretórios com um `agent.py` válido — é o que alimenta `GET /api/agents` e a validação de `agent` no webhook. Levanta `AgentNotFoundError` (tratado globalmente em `server/main.py` como HTTP 400) se o agente não existir.

`agents/catalog/rhawk_assistant/agent.py`

Define `build_graph()` , a factory do agente **rhawk_assistant** usando `create_agent` do LangChain 1.x. Monta: modelo principal (`create_chat_model()`), middleware de contexto conforme `CONTEXT_STRATEGY` (`get_context_middleware()`), tools de memória (`save_memory` / `read_memory` , habilitadas automaticamente se houver `store`), system prompt (`prompts.py`), e os objetos `checkpoint` / `store` recebidos por parâmetro (injetados pela API/Worker em produção, `None` em testes/dev simples).

`agents/catalog/rhawk_assistant/graph.py`

Exporta a variável `graph` exigida pelo `langgraph dev / langgraph.json`. Chama `build_graph(enable_memory_tools=True)` **sem** checkpointer/store customizados, porque a plataforma LangGraph Studio injeta os seus próprios automaticamente — passar um store customizado seria rejeitado no carregamento.

`agents/catalog/rhawk_assistant/prompts.py`

Contém `SYSTEM_PROMPT`: a persona do assistente da comunidade "Top Hawks" — diretrizes de tom (português, direto, respostas curtas para WhatsApp) e instruções explícitas de quando usar `save_memory / read_memory`.

`agents/catalog/rhawk_assistant/__init__.py`, `agents/catalog/__init__.py`, `agents/__init__.py`

Marcadores de pacote Python; `agents/catalog/__init__.py` mantém o diretório `catalog/` importável (necessário para o `importlib.import_module` do loader funcionar).

`agents/middleware/context.py`

Factory `get_context_middleware()` que decide, com base em `settings.context_strategy` (`trim / summarize / none`), qual(is) middleware(s) de gerenciamento de contexto passar ao `create_agent()`. Permite overrides explícitos de todos os parâmetros (útil em testes).

`agents/middleware/trim.py`

Estratégia **trim**: mantém apenas os N turnos mais recentes (`keep_turns`), descartando os antigos via `RemoveMessage` (o reducer `add_messages` do LangGraph faz merge, não replace, então remover mensagens exige emitir `RemoveMessage` explicitamente). Um "turno" é definido como uma `HumanMessage` + tudo até a próxima `HumanMessage` (incluindo tool calls) — nunca corta no meio de um turno. Custo zero (sem chamada LLM extra).

`agents/middleware/summarize.py`

Estratégia **summarize**: usa `SummarizationMiddleware` do LangChain para resumir mensagens antigas quando o histórico excede `trigger_tokens`, mantendo apenas as `keep_messages` mais recentes junto com o resumo. Define `DEFAULT_PROMPT` em português, focado em preservar nome/preferências/decisões e descartar saudações/repetições. Custo extra: uma chamada LLM por sumarização (usa modelo separado, tipicamente mais barato, via `SUMMARIZE_MODEL`).

`agents/middleware/__init__.py`

Reexporta `get_context_middleware`, `create_trim_middleware`, `create_summarize_middleware` para import direto de `whatsapp_langchain.agents.middleware`.

`agents/tools/memory.py`

Duas tools LangChain expostas ao agente:

- `save_memory(memory)` : grava um fato no `store` (namespace `(user_id, "memories")`), com `key` `uuid4()`.
- `read_memory(query, limit)` : busca por similaridade semântica (`store.asearch`) e formata os resultados como lista textual.

Ambas resolvem `store` via `InjectedStore` (LangGraph injeta o store real em runtime) e `user_id` a partir do `configurable` do contexto de execução (`_extract_configurable` / `_extract_namespace`) — o `user_id` é o telefone do remetente, setado pela API/Worker ao invocar o grafo.

`agents/tools/__init__.py`

Reexporta `save_memory` , `read_memory` .

3.3 `server/` — API FastAPI

`server/main.py`

Application factory. Define o `lifespan` (startup: logging → pool Postgres → migrações → bootstrap do schema LangGraph → conexão Redis; shutdown: fecha pool e Redis). Registra `SessionMiddleware` (cookies assinados via `itsdangerous` , para a sessão de admin — adicionado **antes** do CORS para ficar por dentro na pilha) e `CORSMiddleware` (origem única = `settings.frontend_origin` , `allow_credentials=True` para permitir cookies). Trata globalmente `AgentNotFoundError` → HTTP 400. Inclui todos os routers; `admin_router` é o único protegido globalmente por `Depends(require_admin_session)` .

`server/dependencies.py`

FastAPI dependencies reutilizáveis:

- `validate_twilio_signature()` : valida `X-Twilio-Signature` via `RequestValidator` (SDK oficial do Twilio, HMAC-SHA1), usando `build_validation_url()` para reconstruir a URL pública correta mesmo atrás de proxy/túnel (`TWILIO_WEBHOOK_URL`). Só roda se `VALIDATE_TWILIO_SIGNATURE=true` .
- `check_rate_limit(phone_number)` : rate limit distribuído via Redis (sorted set, sliding window de 1h) — compartilhado entre réplicas da API. Levanta HTTP 429 se excedido.
- `require_admin_session()` : exige `request.session["admin_username"]` ; usado por todas as rotas `/api/*` e por `/api/auth/me` .

`server/routes/health.py`

`GET /health` — testa conectividade com o Postgres (`check_db_health`) e retorna `{"status": "ok"|"degraded", "database": ..., "version": ...}` (503 se degradado).

`server/routes/webhook.py`

POST `/webhook/twilio?agent=<id>` — o endpoint de produção. Recebe form-data do Twilio, valida se o `agent` existe, normaliza `From / WaId` para E.164, trata mídia (`NumMedia` , `MediaUrl0` , `MediaContentType0`), aplica rate limit, e chama `enqueue_or_buffer` . Responde **imediatamente** com TwiML vazio (200) — todo o processamento pesado acontece depois, no Worker. Rejeita requests sem identidade de remetente (400) e usa `Depends(validate_twilio_signature)` para autenticidade.

`server/routes/webhook_sync.py`

POST `/webhook/sync?agent=<id>` — endpoint **educacional**, não usado em produção. Processa a mensagem inline (sem fila, sem debounce, sem retry, sem rate limit) e retorna a resposta do agente diretamente na chamada HTTP. Usa `InMemoryStore()` para memória (sem persistência entre chamadas) e nenhum checkpoint (cada request é uma conversa isolada). Serve para testar o fluxo do agente rapidamente via `curl` , sem precisar do Worker rodando.

`server/routes/auth.py`

Rotas `/api/auth/login` , `/api/auth/logout` , `/api/auth/me` . Login compara usuário/senha com `hmac.compare_digest` (evita timing attacks) contra `ADMIN_USERNAME / ADMIN_PASSWORD` do `.env` ; em sucesso, grava `admin_username` na sessão (cookie assinado). Não há tabela de usuários — é um único admin fixo, decisão deliberada para manter o sistema stateless (documentada em `docs/ARCHITECTURE.md`).

`server/routes/admin.py`

Rotas somente-leitura para o Admin Panel, todas exigindo sessão de admin (via `main.py`):

- `GET /api/agents` : lista agentes do catálogo.
- `GET /api/chats` : lista conversas paginadas, ordenadas por `last_message_at` .
- `GET /api/chats/{phone_number}` : histórico de mensagens (`message_queue`) de um telefone específico.
- `GET /api/metrics` : métricas do dia (total, falhas, tempo médio de processamento, tamanho atual da fila).

`server/routes/__init__.py` , `server/__init__.py`

Marcadores de pacote.

3.4 worker/ — processador assíncrono da fila

worker/main.py

Entry point (`python -m whatsapp_langchain.worker.main`). Faz o boot completo: logging → pool Postgres → migrações → abre `checkpointer / store` (uma única vez, reutilizados por todas as mensagens) → valida credenciais Twilio **obrigatórias** (fail-fast com `SystemExit` se faltar `TWILIO_ACCOUNT_SID / API_KEY_SID / API_KEY_SECRET / FROM_NUMBER`) → cria `TwilioClient` → entra em loop infinito: `claim_next_message()` e, se houver mensagem, `process_message()` ; se a fila estiver vazia, dorme `poll_interval_seconds` . No shutdown (`KeyboardInterrupt` ou erro), fecha `store/checkpointer/pool` corretamente via `AsyncExitStack` .

worker/consumer.py

Wrapper fino sobre `shared.queue.claim_next()` com logging contextual do Worker (`queue_empty` em debug quando não há mensagens).

worker/processor.py

O orquestrador central de cada mensagem processada (`process_message`):

1. Pré-processa mídia → texto (`preprocess_incoming_message`).
2. Se mídia falhou/desabilitada, **não invoca o agente** — envia uma resposta automática via Twilio e marca `done` (auditando `media_processing_status / error`).
3. Envia typing indicator (best-effort — falha não interrompe o fluxo).
4. Carrega o grafo do agente (`load_graph`) com o `checkpointer / store` do boot, monta `HumanMessage` com o texto normalizado, invoca com `configurable.thread_id / user_id` .
5. Envia a resposta via Twilio — **isto é obrigatório antes de `mark_done`** : só se marca sucesso após confirmação de envio. Falhas em qualquer etapa caem no `except` geral, que chama `mark_failed` (retry com backoff, decisão de `shared/queue.py`).
6. Em sucesso, também faz `upsert_conversation` para alimentar o Admin Panel.

worker/media.py

Pré-processamento multimodal — a regra de ouro é "**o agente sempre recebe texto**".

`preprocess_incoming_message()` decide, com base em `media_url / media_type` :

- Sem mídia → passa o texto adiante sem alteração.
- Mídia incompleta/tipo não suportado → não invoca o agente, retorna auto-resposta apropriada.
- Imagem/áudio desabilitados via `settings.media_image_enabled / media_audio_enabled` → idem.
- Caso suportado e habilitado → faz `download_media()` (autenticado com API Key do Twilio) e chama o modelo multimodal do OpenRouter (`_describe_image / _transcribe_audio` , via `_chat_completion_media`) para gerar uma descrição/transcrição, que é concatenada ao `body` original como `normalized_text` .

Qualquer exceção durante o download/chamada cai em `media_processing_status="failed"` com auto-resposta de erro — nunca propaga uma falha "crua" ao usuário.

`worker/twilio_client.py`

Cliente HTTP assíncrono (via `httpx`) para a **Twilio Messages API**, usando autenticação por API Key (`api_key_sid` / `api_key_secret` — separada do `auth_token` usado só para validar assinaturas inbound). Dois métodos:

- `send_message(to, body)`: `POST .../Messages.json`, retorna o `sid` da mensagem enviada; levanta `TwilioSendError` em falha HTTP.
- `send_typing(to, message_sid)`: `POST` no endpoint `Indicators/Typing.json` (Public Beta) — marca a mensagem original como lida e mostra "digitando..." por até 25s; falhas são silenciosas (retornam `False`, logadas como warning), pois é um efeito cosmético opcional.

Valida no `__init__` que todos os parâmetros obrigatórios foram fornecidos e que `from_number` está no formato `whatsapp:+...`.

`worker/__init__.py`

Marcador de pacote.

4. Banco de dados — `db/`

`db/migrations/001_initial.sql`

Schema inicial. Habilita a extensão `vector` (pgvector, necessária para o store semântico). Cria:

- `_migrations`: tabela de controle de migrações já aplicadas.
- `message_queue`: a fila principal (campos de identidade, mídia, status, debounce/retry/lease, resposta/erro, timestamps), com três índices — `idx_queue_polling` (parcial, otimizado para o polling do Worker), `idx_queue_phone_agent` (debounce/admin) e `idx_queue_created` (ordenação cronológica).
- `conversations`: agregado por `(phone_number, agent_id)` (UNIQUE), atualizado via UPSERT a cada mensagem processada — alimenta o Admin Panel sem precisar varrer `message_queue`.

`db/migrations/002_media_processing_audit.sql`

Migração incremental: adiciona `normalized_input`, `media_processing_status`, `media_processing_error` em `message_queue`, para auditar o pré-processamento de mídia (o que foi efetivamente enviado ao agente e se houve falha).

`db/migrate.py`

Script standalone (`python db/migrate.py` , também usado por `make migrate`) que aplica as mesmas migrações **fora** do lifespan da aplicação — útil para rodar migrações manualmente/em CI sem subir a API inteira. Usa `psycopg` síncrono e lê `DATABASE_URL` do `.env` /ambiente.

Nota: a lógica de migração está **duplicada** entre `shared/db.py:run_migrations` (async, chamada no lifespan da API/Worker) e `db/migrate.py` (síncrona, standalone) — ambas seguem o mesmo padrão de tabela `_migrations` , então aplicar por qualquer um dos dois caminhos é seguro e idempotente.

5. Frontend — `frontend/` (Admin Panel em Next.js)

Aplicação Next.js 15 (App Router, React 19, TypeScript), servida em modo `standalone` para produção. Não tem CSS framework — estilos inline + `globals.css` .

`frontend/package.json`

Dependências mínimas: `next` , `react` , `react-dom` . Sem bibliotecas de UI/estado — tudo é `useState` / `useEffect` simples. Scripts padrão do Next (`dev` , `build` , `start` , `lint`).

`frontend/next.config.js`

Única configuração: `output: "standalone"` — gera um build autocontido (usado pelo `frontend/Dockerfile` para uma imagem final mínima, sem precisar de `node_modules` completo).

`frontend/middleware.ts`

Proxy same-origin: reescreve (`NextResponse.rewrite`) qualquer request para `/api/*` , `/webhook/*` ou `/health` para `API_PROXY_TARGET` (ex.: `http://api:8000` dentro do Docker, `http://localhost:8000` em dev). Implementado como Middleware (não `next.config.js` rewrites) porque o valor de `API_PROXY_TARGET` só existe em **runtime** dentro do container — rewrites do Next são resolvidos em build time. Isso mantém o cookie de sessão sempre same-site, evitando complexidade de CORS com credenciais.

`frontend/lib/api.ts`

Client HTTP único do frontend (`fetch` com `credentials: "include"` para enviar o cookie de sessão). Define `ApiError` , todas as interfaces TypeScript espelhando as respostas da API (`AdminUser` , `Metrics` , `Chat` , `ChatMessage` , etc.) e o objeto `api` com métodos: `login` , `logout` , `me` , `metrics` , `agents` , `chats` , `chatMessages` . Ponto único de integração com o backend — nenhuma página faz `fetch` diretamente.

`frontend/app/layout.tsx`

Layout raiz (`<html lang="pt-BR">`), define metadata global (título/descrição) e importa `globals.css` .

`frontend/app/globals.css`

Folha de estilos global (cores, `.card` , `.muted` , `.error-text` , tabelas) usada por todas as páginas — não lida em detalhe aqui, mas referenciada por todas as páginas via classes utilitárias.

`frontend/app/page.tsx`

Rota raiz (`/`). Componente client-only que verifica sessão (`api.me()`) e redireciona para `/dashboard` (autenticado) ou `/login` (não autenticado). Não renderiza nada visível — é puro roteamento.

`frontend/app/login/page.tsx`

Formulário de login (usuário/senha) chamando `api.login()` ; em sucesso navega para `/dashboard` ; erros de `ApiError` são exibidos inline.

`frontend/app/(protected)/layout.tsx`

Layout compartilhado pelas rotas autenticadas (route group `(protected)` , não aparece na URL). Verifica sessão no mount; se inválida, redireciona para `/login` . Renderiza uma sidebar fixa (Dashboard/Conversas/Agentes + usuário logado + botão Sair) e a área de conteúdo (`children`).

`frontend/app/(protected)/dashboard/page.tsx`

Página inicial pós-login: busca `api.metrics()` no mount e a cada 10s (`setInterval`), exibindo 4 cartões — mensagens hoje, falhas hoje, tempo médio de processamento, tamanho da fila.

`frontend/app/(protected)/chats/page.tsx`

Lista paginada de conversas (`api.chats` , 20 por página) em tabela — telefone (link para detalhe), agente, última mensagem, data, contagem de mensagens. Paginação client-side via `offset / limit` .

`frontend/app/(protected)/chats/[phone]/page.tsx`

Detalhe de uma conversa: lê `phone` da URL (`useParams`), busca `api.chatMessages(phone)` e renderiza cada mensagem como um card com timestamp, status (`StatusBadge` , colorido por `done / failed /outro`), texto do usuário (`incoming_message` ou `normalized_input`) e resposta do agente (`response` ou `error`).

`frontend/app/(protected)/agents/page.tsx`

Lista simples dos `agent_id` s retornados por `api.agents()` — reflete o catálogo dinâmico do backend (`agents/loader.py:list_agents()`).

`frontend/.env.example`

Template de variáveis de ambiente do frontend (equivalente ao `.env.example` da raiz, mas para o Next.js — tipicamente `API_PROXY_TARGET` / `NEXT_PUBLIC_API_BASE_URL`).

`frontend/public/.gitkeep`

Mantém a pasta `public/` versionada mesmo sem assets estáticos ainda.

`frontend/tsconfig.json` , `frontend/next-env.d.ts`

Configuração padrão do TypeScript para projetos Next.js (paths, strict mode, tipos globais do Next).

`frontend/.next/`

Diretório de build gerado pelo `next build` (ou `next dev`) — artefato, não código-fonte. Contém o output `standalone` , manifests, chunks JS, cache do webpack, etc. Não deveria ser versionado nem editado manualmente.

6. Testes — `tests/`

`tests/conftest.py`

Fixtures globais do pytest: carrega `.env` , define `sample_messages` (conversa de exemplo para testar middlewares de contexto) e `live_openrouter_api_key` (fixture que faz `pytest.skip` a menos que `OPENROUTER_LIVE_TESTS=1` e uma chave real esteja configurada — protege contra rodar testes que consomem API paga acidentalmente).

`tests/unit/` — **testes unitários (sem dependências externas, rodam em `make test`)**

- `test_debounce.py` — lógica de agrupamento de mensagens (`enqueue_or_buffer`).
- `test_llm_factory.py` — `shared/llm.py` (criação/cache de rate limiters).
- `test_loader.py` — `agents/loader.py` (carregamento dinâmico, `AgentNotFoundError`).
- `test_media_preprocess.py` — `worker/media.py` (decisões de habilitar/desabilitar mídia, mocks de download/transcrição).
- `test_memory_flag.py` — comportamento quando `MEMORY_ENABLED=false` .
- `test_memory_tool.py` — `agents/tools/memory.py` (`save_memory` / `read_memory` com store fake).
- `test_models.py` — validação dos modelos Pydantic (`shared/models.py`).
- `test_processor_twilio.py` — `worker/processor.py` (fluxo de envio Twilio obrigatório antes de `mark_done`).
- `test_queue_claim.py` — `claim_next` (concorrência, `FOR UPDATE SKIP LOCKED` , lease expirado).
- `test_queue_retry.py` — `mark_failed` (backoff progressivo, falha definitiva).

- `test_rate_limit.py` — `check_rate_limit` (sliding window Redis, provavelmente com `fakeredis`).
- `test_twilio_client.py` — `TwilioClient` (envio de mensagem/typing, tratamento de erro).
- `test_twilio_signature.py` — `validate_twilio_signature` (HMAC válido/inválido).

`tests/integration/` — testes de integração (alguns exigem stack real)

- `helpers.py` : utilitários E2E — cliente admin autenticado, geração de telefones/SIDs únicos, queries diretas ao Postgres (`query_message_status` , `query_conversation` , `memory_count_for_user`), funções de espera por polling (`wait_terminal_status` , `wait_memory_saved` , `wait_conversation_count` , `wait_queue_done`) e `send_webhook` / `send_webhook_and_wait` (simulam requests do Twilio). Usado pelos testes marcados `docker_demo` .
- `test_auth.py` — login/logout/sessão do admin panel.
- `test_context_middleware.py` — `trim` / `summarize` end-to-end (parte marcada como live, chamando OpenRouter real).
- `test_docker_demo_features.py` — cenários demonstrativos que exigem `make up` rodando (marker `docker_demo`).
- `test_media_real.py` — processamento real de imagem/áudio via OpenRouter (usa `tests/assets/sample.png` / `sample.ogg`), só roda com `OPENROUTER_LIVE_TESTS=1` .
- `test_memory.py` — memória semântica ponta a ponta (save → recall), inclui casos live.
- `test_realistic_flows.py` — fluxos completos simulando conversas reais via webhook (usa `helpers.py`).
- `test_webhook.py` — `/webhook/twilio` e `/webhook/sync` (validação de payload, enfileiramento, resposta TwiML).

`tests/assets/sample.ogg` , `tests/assets/sample.png`

Arquivos de mídia reais usados por `test_media_real.py` para testar transcrição de áudio e descrição de imagem contra a API OpenRouter de verdade.

`tests/stress/locustfile.py`

Script Locust que simula usuários reais enviando mensagens ao `/webhook/twilio` (pool fixo de 50 telefones para exercitar debounce/rate-limit de forma realista, não usuários "novos" a cada request). **Só deve rodar contra a stack local** com `VALIDATE_TWILIO_SIGNATURE=false` — o Locust não consegue gerar uma assinatura HMAC-SHA1 válida do Twilio.

`tests/stress/README.md`

Instruções de uso do stress test (modo UI vs headless), complementando os comandos `make stress` / `make stress-headless` .

`tests/__init__.py` , `tests/integration/__init__.py` , `tests/unit/__init__.py`

Marcadores de pacote (permitem imports relativos entre arquivos de teste).

7. Documentação — `docs/`

`docs/ARCHITECTURE.md`

O documento mais denso do projeto — arquitetura completa: fronteiras entre API/Worker/Shared, modelo de dados (`message_queue` , `conversations`), fluxo end-to-end (10 passos, do usuário no WhatsApp até a resposta), contexto vs. memória (checkpointer vs. store), controles operacionais (debounce, retry/backoff, rate limits, autenticação do admin), endpoints disponíveis e decisões arquiteturais justificadas (por que Postgres como fila, por que API/Worker separados, por que sessão via cookie e não JWT, etc.).

`docs/GETTING_STARTED.md`

Guia de primeiros passos: pré-requisitos, setup local, duas trilhas de desenvolvimento (Trilha A: iterar no agente via LangGraph Studio; presumivelmente Trilha B: rodar a stack completa).

`docs/ADDING_AGENTS.md`

Como criar um novo agente no catálogo: contrato obrigatório (`build_graph()` em `agent.py`), regras e passo-a-passo (criar diretório, `agent.py` , `prompts.py` , opcionalmente `graph.py` para LangGraph Studio).

`docs/DATABASE.md`

Referência de schema: visão geral das tabelas de aplicação (`_migrations` , `message_queue` , `conversations`) e presumivelmente das tabelas internas do LangGraph (checkpoints, store).

`docs/TWILIO.md`

Guia extenso de integração com o Twilio do zero: criar conta, obter credenciais (Account SID, Auth Token, API Key), configurar o Sandbox do WhatsApp, expor o webhook local via `cloudflared` (túnel), e configurar `TWILIO_WEBHOOK_URL` corretamente.

`docs/DEPLOY.md`

Guia de deploy em produção: escopo da fase atual, topologia (Docker Compose + Caddy), variáveis obrigatórias, ordem de subida dos serviços — complementa `docker-compose.prod.yml` e `Caddyfile` .

`docs/guia-debug-manual.md`

Runbook prático para testar/debugar o sistema manualmente: subir a stack, abrir o Swagger UI (`/docs` do FastAPI), enviar mensagens via webhook e inspecionar o resultado passo a passo. Útil para depuração sem escrever testes automatizados.

`docs/architecture.png`

Diagrama visual referenciado por `README.md` e `docs/ARCHITECTURE.md` ([!Arquitetura](#)).

8. Scripts operacionais — `scripts/`

`scripts/backup_db.sh`

Script bash (`make backup`) que roda `pg_dump` **dentro do container** `db` via `docker compose exec` , comprime com `gzip` e salva em `backups/` (não versionado) com timestamp no nome. Aplica retenção configurável (`BACKUP_RETENTION_DAYS` , default 14 dias), apagando backups mais antigos. Comentários no topo do arquivo já documentam o comando de restauração e um exemplo de agendamento via `crontab` .

9. Fluxo de dados resumido (para amarrar tudo)

1. **Webhook** (`server/routes/webhook.py`) recebe o POST do Twilio, valida assinatura (`server/dependencies.py`), aplica rate limit (Redis) e chama `enqueue_or_buffer` (`shared/queue.py`) — que insere ou concatena (debounce) uma linha em `message_queue` . Responde 200 imediatamente.
 2. **Worker** (`worker/main.py`) faz polling contínuo via `claim_next` (`shared/queue.py`), que reserva atomicamente a próxima mensagem elegível.
 3. `worker/processor.py` orquestra: pré-processa mídia (`worker/media.py` , chamando OpenRouter multimodal se necessário), envia typing (`worker/twilio_client.py`), carrega o agente (`agents/loader.py` → `agents/catalog/rhawk_assistant/agent.py`) com o `checkpointer` / `store` abertos no boot do Worker, invoca o grafo (aplica middleware de contexto e pode chamar as tools de memória), e envia a resposta via Twilio.
 4. Só após confirmação de envio, `mark_done` / `upsert_conversation` (`shared/queue.py`) persistem o resultado — alimentando as tabelas que o **Admin Panel** (`frontend/`) consulta via `server/routes/admin.py` .
 5. Em qualquer falha no meio do caminho, `mark_failed` decide entre reagendar (retry com backoff) ou marcar como `failed` definitivamente.
-

Gerado a partir da leitura direta do código-fonte do repositório em 2026-09-10. Para o estado mais atual, prefira sempre o código e `git log` — este documento pode ficar desatualizado conforme o projeto evolui.